

fontana_escritorio

Versión de escritorio (Python + Tkinter) de fontana_movimientos

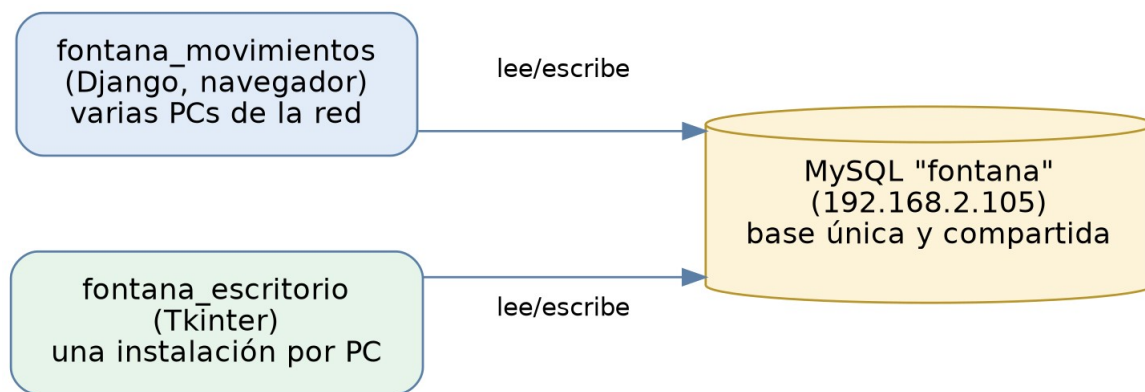
Documento de referencia — 8 de septiembre de 2026

1. Qué es y por qué existe

fontana_movimientos es un sistema grande: 13 apps de Django (entidades, productos, movimientos, movimientos_caja, comprobantes, retenciones, retenciones_inym, liquidaciones, respaldo, tipos, solicitudes_compra, empleados, remitos), con MySQL como base y acceso por navegador desde varias PCs de la red.

fontana_escritorio es la versión de ese mismo sistema como aplicación de escritorio, hecha en Python con Tkinter. No es una copia ni un sistema aparte: se conecta DIRECTO a la misma base MySQL de producción, como un cliente más -- los mismos datos, en tiempo real, en paralelo con la web.

Migrar los 13 módulos de una sola vez no es realista en una sola sesión de trabajo, así que el proyecto se arma módulo por módulo, reusando siempre las mismas tablas y las mismas reglas de negocio que ya tiene la versión Django, para no duplicar lógica que después se desincroniza entre las dos versiones.

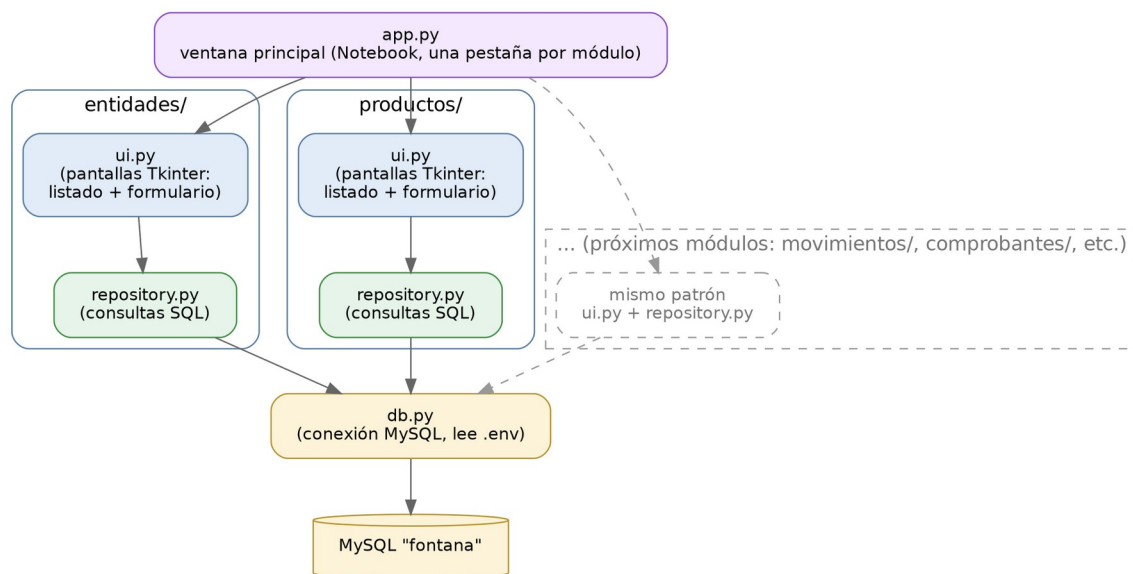


Ambas versiones (web y escritorio) conviven y comparten la misma base.

2. Arquitectura interna

Cada módulo (Entidades, Productos, y los que se vayan agregando) sigue siempre el mismo patrón de dos archivos, para que el acceso a datos se pueda revisar o probar por separado de la interfaz gráfica:

- **repository.py**: las consultas SQL (listar, obtener, crear, actualizar, dar de baja/eliminar). No conoce nada de Tkinter.
- **ui.py**: las pantallas (listado con Treeview + formularios en ventanas emergentes). Llama al repository correspondiente y muestra los resultados o errores.



app.py arma la ventana principal (una pestaña por módulo); db.py centraliza la conexión a MySQL, leyendo las credenciales de un archivo .env.

3. Decisiones de diseño

- **Sin login propio:** por ahora esta app no pide usuario/contraseña (Django sí lo tiene). Si hace falta restringir el acceso igual que en la web, se puede sumar más adelante.
- **IDs de tablas legadas asignados a mano:** "entidad" y "producto_detalle" no se tratan como autoincrementales -- se calcula $\text{MAX}(\text{id})+1$ antes de cada alta, igual que hace el propio proyecto Django para estas mismas tablas, para no arriesgarse a chocar con un id ya usado si la columna no fuera realmente AUTO_INCREMENT.
- **Baja de Entidad = activo=0, nunca DELETE:** mismo criterio que la web -- no se borra una entidad, se la marca inactiva y se puede reactivar.
- **Eliminar Producto sí es DELETE real,** pero con un chequeo previo de uso (movimientos, renglones de comprobante, renglones de remito) y, además, si la base rechaza el borrado por una relación real, se captura ese error y se avisa en vez de que la app se caiga. El chequeo de uso es una réplica hecha desde afuera del código Django real (productos.views.producto_eliminar); conviene compararlo contra esa vista si se quiere blindar del todo.

4. Estado actual

Módulo	Estado	Notas
Entidades	✓ Hecho	Listado con búsqueda y filtro de inactivas, alta, edición, dar de baja/reactivar.
Productos	✓ Hecho	Listado con búsqueda, alta, edición, eliminar con chequeo de uso.
Movimientos	□ Pendiente	Próximo módulo a implementar.
Comprobantes	□ Pendiente	Lo usan Liquidaciones y Cuenta Corriente de Productos.
Movimientos de Caja	□ Pendiente	
Retenciones / Retenciones INYM	□ Pendiente	
Liquidaciones	□ Pendiente	
Remitos	□ Pendiente	

Módulo	Estado	Notas
Cuenta Corriente de Productos	☐ Pendiente	Ya está terminado del lado Django (ver documento aparte); falta portarlo acá.
Solicitudes de Compra	☐ Pendiente	
Empleados	☐ Pendiente	
Tipos	☐ Pendiente	
Respaldo	☐ A evaluar	Es un módulo de backup del servidor; puede no tener sentido igual en un cliente de escritorio.

5. Módulos terminados

5.1 Entidades

- Listado con búsqueda por nombre o CUIT, y checkbox para mostrar también las inactivas.
- Alta y edición con todos los campos: nombre, CUIT, dirección, documento N°, código, localidad, código postal, IVA, provincia.
- Dar de baja / reactivar (nunca elimina, ver decisiones de diseño).
- Doble clic en una fila abre la edición directamente.

5.2 Productos

- Listado con búsqueda por nombre, mostrando también el tipo de ítem (Producto, Servicio, etc.).
- Alta y edición: nombre + tipo (combo desplegable con los item_tipo existentes).
- Eliminar, bloqueado si el producto está en uso.

6. Instalación y uso

Requisitos: Python 3 con Tkinter (incluido en la instalación estándar de Windows) y las dependencias del archivo requirements.txt.

```
cd fontana_escritorio
pip install -r requirements.txt
copy .env.example .env
    (completar .env con los mismos datos que fontana_movimientos\.env)
python app.py
```

El archivo .env no se genera automáticamente a partir del de Django por seguridad (para no manipular la contraseña real desde afuera) -- alcanza con copiar el archivo existente a esta carpeta.

7. Cómo se probó

Antes de entregar el código de Entidades y Productos se corrió una prueba automática con una pantalla virtual (Xvfb): se abrió la ventana principal, se simularon altas, ediciones, bajas/reactivaciones y el bloqueo de "eliminar producto en uso" con datos de prueba (sin tocar la base real), confirmando que ninguna de esas acciones tira error y que las pantallas se ven correctamente.

8. Estructura de carpetas

```
fontana_escritorio/
├── app.py          (ventana principal)
├── db.py           (conexión a MySQL, lee .env)
```

```
├── requirements.txt
├── .env.example
├── entidades/
│   ├── repository.py
│   └── ui.py
├── productos/
│   ├── repository.py
│   └── ui.py
```

9. Roadmap confirmado

Orden acordado para seguir agregando módulos, pensado por dependencias (Movimientos y Comprobantes necesitan los catálogos de Entidades/Productos, que ya están listos). Se puede reordenar en cualquier momento.

- 1. Movimientos -- el módulo de uso más frecuente día a día.
- 2. Comprobantes -- lo usan Liquidaciones y Cuenta Corriente de Productos.
- 3. Movimientos de Caja
- 4. Retenciones + Retenciones INYM
- 5. Liquidaciones
- 6. Remitos
- 7. Cuenta Corriente de Productos
- 8. Solicitudes de Compra
- 9. Empleados
- 10. Tipos

(Respaldo queda a evaluar aparte, ver tabla de estado.)